



Facultad de Ingeniería - Udelar

Práctico 2

Grillas y acceso coalesced

Manuel Roca

Belén Olivera

Nicolás Pereira

Ejercicio 3 - Memoria Global

Parte 3.1 - Bloque 32×32

Para la transposición de matrices en memoria global, el kernel que implementamos calcula los índices de la siguiente forma:

```
int col = blockIdx.x * blockDim.x + threadIdx.x;
int row = blockIdx.y * blockDim.y + threadIdx.y;
output[col * N + row] = input[row * N + col];
```

Análisis del patrón de acceso:

Lecturas (matriz de entrada): Los 32 hilos de un warp acceden a posiciones consecutivas en memoria dado que `row` es constante dentro del warp y `col` varía de 0 a 31. Esto resulta en accesos **coalesced**, siguiendo el almacenamiento row-major en memoria.

Escrituras (matriz transpuesta): Los hilos escriben en posiciones separadas por N elementos ($\text{stride} = N$), ya que `col` varía mientras `row` es constante. Esto genera accesos **non-coalesced**.

Ejemplo concreto (para $N = 5$, bloque (0, 0), warp 0):

Hilo	threadIdx	row	col	índice entrada	índice destino
0	(0,0)	0	0	0	0
1	(1,0)	0	1	1	5
2	(2,0)	0	2	2	10
3	(3,0)	0	3	3	15
4	(4,0)	0	4	4	20

Como se observa, las lecturas son consecutivas (0, 1, 2, 3, 4, ...) generando una única transacción de memoria eficiente, mientras que las escrituras tienen un stride de N (0, 5, 10, 15, 20, ...). **Tiempo de ejecución medido:** 2,9 ms

Parte 3.2 - Optimización del tamaño de bloque

Tamaño de bloque elegido: 32×8

Se mantuvo el valor de `blockDim.x = 32` para preservar los accesos coalesced en las lecturas de la matriz origen, dado que un warp siempre ejecuta 32 hilos con `threadIdx.x` consecutivo. En la configuración inicial de 32×32 (1024 hilos), cada bloque cuenta con 32 warps. Al reducir el tamaño a 32×8 , cada bloque cuenta con 8 warps.

Justificación

Al utilizar bloques de 256 hilos en lugar de 1024, se permite que una mayor cantidad de bloques entren simultáneamente en cada SM. Dado que la unidad de ejecución es el warp y estos se gestionan de forma independiente, tener más bloques activos proporciona un conjunto más diverso de warps para ejecutar. De esta manera, cuando los warps de un bloque quedan en espera por la alta latencia de las escrituras no-coalesced en memoria global, el SM tiene warps de otros bloques para despachar. Entendemos que esta flexibilidad mejora el ocultamiento de latencia de la GPU.

Comparación de tiempos: Para la medición usamos matrices de 2048 generadas aleatoriamente.

Tamaño de bloque	Tiempo de ejecución
32×32	2,6518 ms
32×8	2,0141 ms

Conclusión: La mejora observada es poca porque el problema fundamental de las escrituras no-coalesced persiste. Creemos que la solución óptima necesita el uso de memoria compartida para reordenar los datos antes de escribirlos de forma coalesced.